BLOCKCHAIN LABORATORY

Experiment No. 5

Study of Solidity Program Structure, Basic Constructs, Compilation and Deployment Environment

Name: Nikhil Sunchu

Roll No.: 31

Class/section: D20A

Date: _____

1. Aim

Study the structure of a Solidity program, its basic constructs, and the compilation and deployment environment using Remix IDE.

2. Theory

2.1. 1. Solidity and Smart Contracts

Solidity is a high-level, statically typed programming language used to develop smart contracts for Ethereum-compatible blockchain platforms. A smart contract is a program deployed on a blockchain that executes predefined logic when its functions are called. Remix IDE provides an online environment to write, compile, deploy, and test Solidity contracts.

2.2. 2. Structure of a Solidity Program

A Solidity source file commonly contains an SPDX license identifier, a compiler-version pragma, a contract declaration, state variables, and functions. The `pragma` statement specifies the compiler versions compatible with the source code. A contract groups related state and behaviour under one name.

2.3. 3. Data Types, State Variables and Mappings

Solidity provides types such as `uint` for unsigned integers, `bool` for Boolean values, `address` for Ethereum addresses, and `string` for text. State variables are stored as part of the contract's persistent state. A mapping stores key-value pairs and is useful for looking up a value using a key. In this experiment, a mapping associates an Ethereum address with an unsigned integer balance.

2.4. 4. Functions and Visibility

Functions contain the operations performed by a contract. The `public` visibility specifier allows a function to be called both internally and externally. A function marked `view` can read contract state without modifying it. A function that changes a state variable is executed as a transaction when called externally.

2.5. 5. Compilation and Deployment

Compilation checks the Solidity source code and generates contract artifacts, including the ABI and bytecode. Deployment creates a contract instance in the selected blockchain

environment. Remix VM provides a simulated blockchain and test accounts, allowing contracts to be deployed and tested without using real Ether.

2.6. 6. Transactions and Gas

A state-changing function call is submitted as a transaction and consumes gas for its execution. Remix displays information such as the transaction hash, block number, gas used, and execution cost. A read-only `view` function is generally invoked as a call and does not create a state-changing transaction.

3. Tools and Environment Used

- Remix IDE (version 2.6.6 as shown in the screenshots)
- Solidity compiler, version 0.8.34 in the compiler panel
- Remix VM with the Osaka VM setting
- Solidity source file: `Nikhilsunchu.sol`
- Browser-based code editor, compiler, and Deploy & Run Transactions panel

4. Tasks Performed

4.1. Task 1 – Write the Solidity Program

- Created a Solidity source file named `Nikhilsunchu.sol` in Remix IDE.
- Defined the `GasTransaction` contract.
- Used a mapping to store a balance for each Ethereum address.
- Implemented `setBalance()` to update the balance associated with the caller.
- Implemented `getBalance()` to read and return the caller's balance.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.18;

contract GasTransaction {
    mapping(address => uint) public balances;

    function setBalance(uint amount) public {
        balances[msg.sender] = amount;
    }

    function getBalance() public view returns (uint) {
        return balances[msg.sender];
    }
}
```

4.2. Task 2 – Compile the Contract

- Opened the Solidity Compiler panel in Remix IDE.
- Selected a compatible Solidity 0.8.x compiler.
- Compiled the source file by clicking the Compile button.
- Confirmed successful compilation before proceeding to deployment.
- The source code and compiler panel are shown in Figure 1.

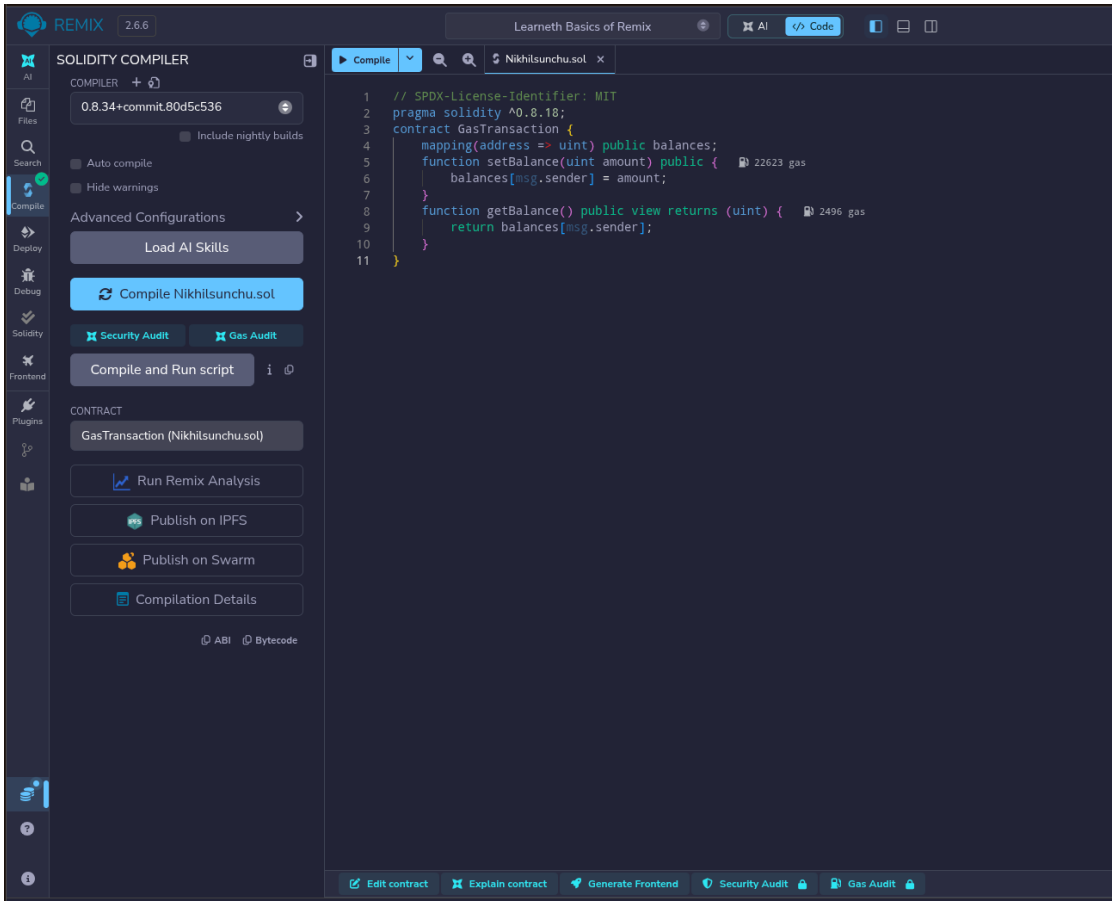


Figure 1: Solidity source code and compiler panel in Remix IDE

4.3. Task 3 – Deploy the Contract

- Opened the Deploy & Run Transactions panel.
- Selected `Remix VM` as the execution environment.
- Selected and deployed the `GasTransaction` contract.
- Verified that the deployed contract appeared in the Deployed Contracts section.
- Used the displayed contract functions for interaction and testing.

4.4. Task 4 – Execute the State-Changing Function

- Entered `500` as the argument to `setBalance()`.
- Clicked the Transact button to execute the function.
- Remix created and mined a transaction because the function modifies contract state.
- Checked the transaction status, transaction hash, block hash, block number, and gas information.
- The successful transaction details are shown in Figure 2.

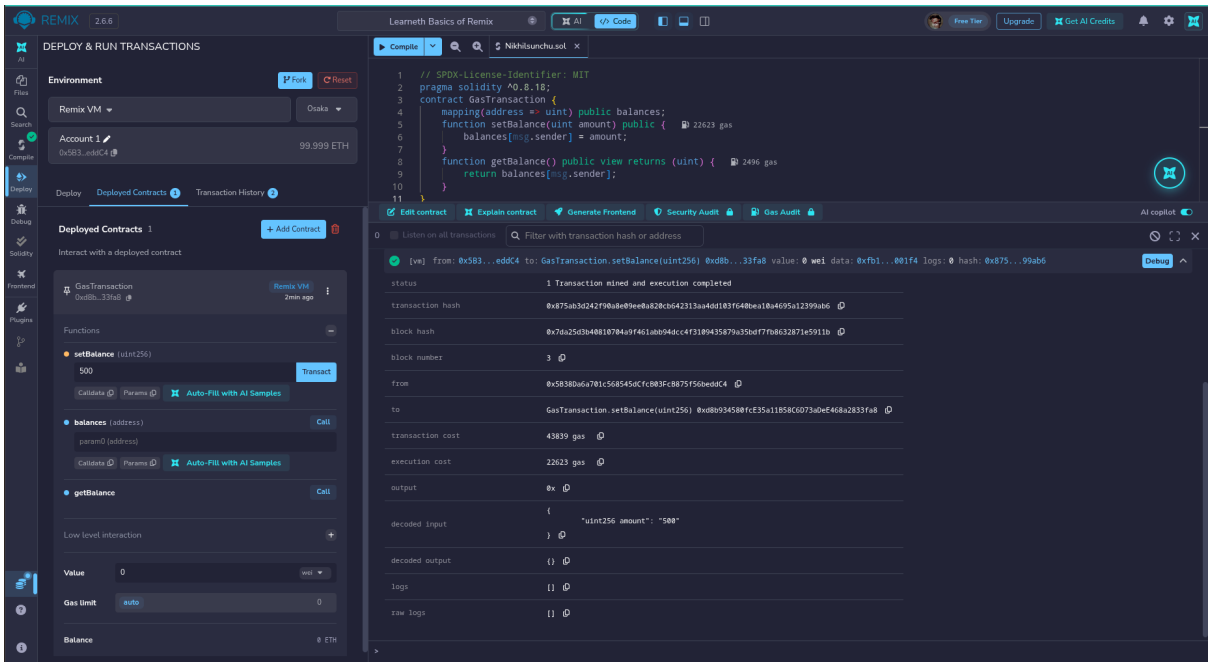


Figure 2: Successful execution of setBalance(500) and transaction details

4.5. Task 5 – Verify the Stored Value

- Called the `getBalance()` function from the deployed contract.
- Observed the returned value `500`.
- Confirmed that the state-changing transaction updated the mapping for the selected account.
- The returned value and Remix terminal output are shown in Figure 3.

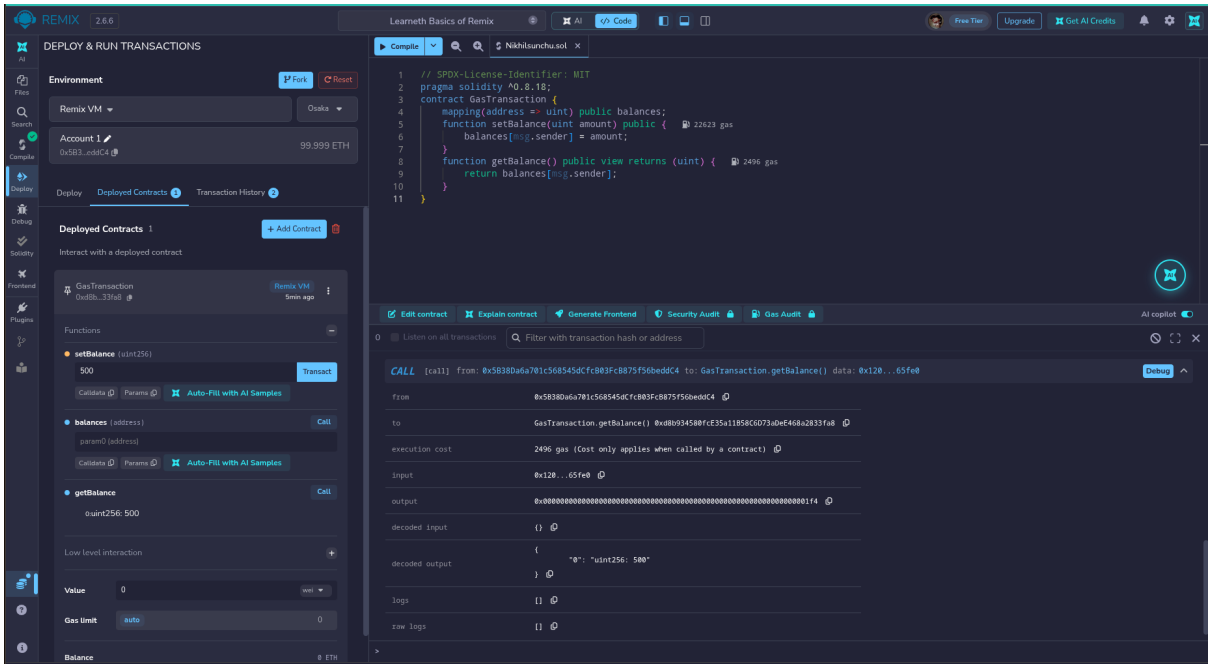


Figure 3: Calling getBalance() and verifying the stored value

5. Result

The Solidity contract was compiled and deployed successfully in Remix VM. The `setBalance(500)` transaction updated the mapping for the caller's address, and `getBalance()` returned `500`. The experiment demonstrated the Solidity contract structure, state variables, mappings, function visibility, read-only functions, deployment, and transaction gas information.

6. Conclusion

The experiment provided practical familiarity with writing and compiling a Solidity smart contract in Remix IDE and deploying it in a simulated blockchain environment. By interacting with the `GasTransaction` contract, the operation of a mapping, a state-changing function, and a `view` function was observed. The transaction details also illustrated how Remix reports execution status and gas usage.